FORWARD DEPLOYED ENGINEER / CANDIDATE ASSIGNMENT

Turning App Ratings into a Better Employee Commute

A practical plan to understand negative reviews, prioritise fixes, and build a repeatable rating-monitoring process.

Prepared by	Anusha Upadhyay
Role	Forward Deployed Engineer (Intern to FTE)

CORE RECOMMENDATION

Fix the failures that make employees lose trust first. Measure whether those fixes reduce new negative reviews, then use review trends as a weekly signal for product, engineering, and operations.

1. Executive summary

MoveInSync supports an important, time-sensitive part of an employee's day: getting to and from work. Employees rely on the app to book transport, confirm their ride, track a vehicle, and complete safety-related boarding steps. When one of these steps fails, the impact can be much bigger than an ordinary app inconvenience.

My recommendation is to treat Play Store reviews as an early-warning system for the complete commute experience. Start by validating the strongest signals, fix critical booking and login failures first, improve everyday usability next, and connect transport-related complaints to the teams that can resolve them.

PRIORITY	WHAT TO DO	WHY FIRST
1 · Reliability	Investigate booking-status mismatches, OTP delivery and boarding-flow problems.	These can prevent an employee from accessing or trusting a ride.
2 · Usability	Review shuttle booking steps, tracking access, check-in reminders and no-show flows.	These are repeated tasks where friction can become a recurring complaint.
3 · Resolution	Route complaints to product, engineering, support or transport operations; close the loop with users.	Not every complaint originates in the app, but each needs an owner.
4 · Measurement	Track new-review trends alongside booking, OTP, crash and support metrics.	The overall rating moves slowly and does not diagnose root causes by itself.

What success should look like

- Fewer booking and login failures in the actual product journey.
- A lower share of new one-star reviews, especially for issues that have been fixed.
- Faster detection of release regressions and clearer ownership of recurring complaints.
- A weekly scorecard that can answer "How are our ratings?" without restarting the research each time.

2. What the preliminary evidence suggests

The initial research notes describe a large employee-app rating base and several recurring complaint types. However, the current headline score is inconsistent across the supplied observations (approximately 4.3 in one view and 4.6 in another). The baseline must be checked against the current Play Store listing and, if available, Play Console before it is used in a final performance claim.

INITIAL SIGNAL	WHAT IT MAY INDICATE	HOW TO VALIDATE
Booking or login uncertainty	Users may not know whether a ride is confirmed, or may be blocked by OTP/login issues.	Check booking API states, OTP delivery and verification logs, and matching review dates/app versions.
Interface changes	A familiar task may have become harder after an update.	Compare review timing with releases; run task-based usability checks with employees.
Tracking / ETA complaints	Location updates or pickup timing may not meet employee expectations.	Compare reported trips with GPS freshness, ETA accuracy, and pickup-point data.
No-show / check-in confusion	Reminder timing or status rules may be unclear or unreliable.	Review event logs, notification delivery, and the exact rules used to mark no-shows.
Support / route complaints	Some dissatisfaction may come from transport operations rather than the app UI.	Categorise tickets and route/driver incidents; compare with reviews and resolution times.

Evidence quality: The initial theme review covered a small manually coded sample (23 reviews). These themes are useful hypotheses, not a representative estimate of all users. Rating figures and calculations should be verified before being presented as facts.

3. Recommended actions, in priority order

I would avoid a broad redesign as the first move. Instead, I would investigate and fix the highest-impact failures that can stop an employee from completing a critical task.

Priority 1 — Make booking and boarding dependable

• **Reproduce the booking-status mismatch.** Check whether a successful backend booking can still trigger a “trip not booked” message. Align the user-facing state with the authoritative booking state.

• **Trace OTP failures end to end.** Measure request, delivery, verification and retry outcomes; separate delivery-provider issues from app or backend failures.

• **Inspect the boarding sequence.** Confirm that seat-belt verification does not hide or block access to the OTP when the employee needs it.

• **Prevent duplicate retries.** Make retry actions safe and provide clear states such as Confirmed, Pending, Failed and Cancelled, based on actual system status.

Acceptance checks: a confirmed booking is never shown as unconfirmed; failed or delayed OTPs have a clear recovery path; critical boarding information remains accessible; and retrying does not create duplicate bookings.

Priority 2 — Reduce friction in repeated tasks

• Map the current shuttle-booking flow and remove unnecessary steps only after confirming where users get stuck.

• Check whether live vehicle location and pickup details are easy to find and update reliably.

• Test the app under weak-network conditions and across commonly used devices.

• Make check-in and no-show rules understandable, with timely reminders and clear explanations when a status changes.

Acceptance checks: improved booking completion and task time; fewer tracking-related complaints; fewer avoidable no-show disputes; and no regression in existing core flows.

Priority 3 — Make complaint resolution visible

• Classify complaints into app defects, booking/notifications, tracking, transport operations, and support.

• Assign each category to a named team or owner and track it in a shared issue list.

• Acknowledge negative reviews respectfully and offer a relevant support route without arguing with the user.

• Use recurring complaint patterns to decide whether the root cause is product, process, driver/route operations, or communication.

Acceptance checks: improved time to first response and resolution, fewer repeat complaints, and clear ownership for the most frequent unresolved issues.

4. First 30 days: execution plan

WHEN	ACTION	OWNER(S)	OUTPUT / DECISION
Days 1–3	Confirm live rating, app identity, rating count and recent-review sample. Establish a baseline for key product metrics.	FDE / Product / Data	Validated baseline and evidence log.
Days 1–5	Reproduce booking confirmation and OTP complaints; inspect relevant logs, app versions and support tickets.	Engineering / QA / Support	Ranked defects with reproduction steps and severity.
Week 2	Fix the highest-impact verified defect(s); add regression tests and instrument key failure points.	Engineering / QA	Tested fix with measurable acceptance criteria.
Weeks 2–3	Test booking-flow, tracking and check-in improvements with a small employee group.	Product / UX / QA / Operations	Usability findings and go/no-go decision.
Weeks 3–4	Release gradually; watch reliability metrics and negative-review themes; pause rollout if agreed guardrails are breached.	Engineering / Product / Operations	Safer rollout and documented outcome.
Weekly	Review scorecard, top complaint themes, owners and overdue actions.	Product + Engineering + Operations	Updated priorities and follow-up actions.

Release guardrail: For changes affecting booking or boarding, use a staged rollout where possible. Agree on thresholds before release for booking failure, OTP success, crash rate and support contacts. If a critical metric worsens materially, pause expansion and investigate rather than waiting for the overall rating to move.

5. How I would measure whether it works

The Play Store rating is an important outcome, but it is a lagging indicator. I would pair it with metrics that show whether the employee journey is actually improving.

METRIC	WHY IT MATTERS	REVIEW CADENCE
Overall Play Store rating and rating count	Tracks public perception, but changes slowly and may differ by store view.	Weekly; confirm against Play Console where available.
New-review average and one-star share	Shows whether recent feedback is improving faster than the lifetime score.	Weekly; use rolling 7- and 30-day windows.
Booking success / failure rate	Direct measure of whether employees can complete the core task.	Daily during release; weekly otherwise.
OTP delivery and verification success	Detects a key access and boarding dependency.	Daily and after relevant changes.
Crash-free sessions	Indicates basic app stability.	Daily during rollout.
Tracking freshness and ETA accuracy	Measures whether users can make decisions using vehicle information.	Daily/weekly, depending on available instrumentation.
Support contacts per 1,000 trips	Connects product friction and operations problems to real trip volume.	Weekly.
Time to first response / resolution	Shows how quickly the company responds to problems it can control.	Weekly.

Rating improvement: set a baseline before setting a target

The preliminary notes suggest a meaningful one-star tail, but the current headline rating and histogram need reconciliation. I would not promise a fixed increase in the displayed score until the true distribution, recent review volume and store rating calculation are confirmed.

I would set initial targets after the first week of validated data. For example, the team could aim to reduce the share of new one-star reviews for the top verified defect categories over the next 30 days, while also improving the relevant product metric.

Analysis rule: compare the latest 30 days with the preceding 30 days and inspect the latest 7 days for early signals after releases. Do not infer that a fix caused a rating change from timing alone; compare product metrics, app versions, review themes and incident records.

6. Make “How are our ratings?” a repeatable answer

I would maintain a simple scorecard that can be refreshed weekly. Its purpose is not only to report a number; it should help the team decide what to investigate and who should act.

SCORECARD SECTION	INCLUDE
Headline	Current rating, total ratings, app/package identity, date checked and source.
Recent movement	New reviews, average stars and one-star share for 7- and 30-day windows versus the prior period.
Issue themes	Common negative-review themes, example excerpts where permitted, counts, confidence and manual spot-check results.
Release signal	Review volume and rating by app version where sample sizes are sufficient; link to release dates.
Response & ownership	Developer reply rate and response time if reliably available; issue owner, status, next action and due date.

Suggested workflow

Collect → **Categorise** → **Validate** → **Prioritise** → **Assign** → **Measure.** Automated keyword tags can speed up triage, but they should not be treated as ground truth. A review can mention multiple problems; a simple first-match approach may assign only one category and miss others. I would manually check a sample every week and refine categories based on false matches and the “other” bucket.

7. Methodology and tools used

I approached the case study by starting with the employee journey described in the brief, then using public information and an initial sample of reviews to identify points where a poor experience could affect trust. I grouped the initial complaints into booking/login, usability, tracking, check-in/no-show, and transport/support themes. I used those themes to propose a priority order, an action plan, and metrics that would help verify whether the changes work.

TOOL / SOURCE	HOW IT WAS USED / INTENDED USE
Google Play Store and public app-store information	Review the public rating and user feedback; current rating and counts must be rechecked before final reporting.
MoveInSync public help/product information and competitor pages	Understand the commute journey and add context to the user experience; competitor comparisons should be verified and used cautiously.
Python + google-play-scraper	Build a repeatable way to retrieve recent reviews and metadata, subject to live access and package availability.
pandas	Organise review data and calculate time-window, version and theme summaries.
AI assistance	Help organise research, structure the analysis and refine the recommendation; findings still require human verification.

Limitations and assumptions

• The review themes are based on preliminary sampling and should not be treated as representative of all users.

• The initial rating figures conflict (approximately 4.3 versus 4.6 in the notes). The current figure must be confirmed before quoting it as fact.

• The monitoring script has been run in demo mode only; live data collection and calculations need validation.

• Keyword matching is indicative, not definitive. A review may contain more than one issue, and automatic labels need manual spot-checking.

• Public reviews cannot show the full cause of an incident. Internal booking, OTP, GPS, crash, support and trip data would be needed to confirm root causes.

Final recommendation

Start by validating and fixing the failures that make employees uncertain about their ride. Then reduce friction in repeated tasks and create a weekly loop connecting public feedback to product and operations data. The goal is not simply to ask for more five-star ratings; it is to give employees fewer reasons to leave a one-star review.